

BCC2001

— ORIGIN  
'warn'.

**monitor** • n.

thing. 2 a software  
duties. 3 a television  
picture for use in  
rooms where

Diego Bertolini

diegobertolini@utfpr.edu.br

# Semana Passada

- Desvios condicionais:

- if-else
- switch
- ?

# Aulas desta Semana

## Comandos de Repetição:

- Definir um laço ;
- Identificar um laço infinito ;
- Utilizar o comando **while** ;
- Utilizar o comando **for** ;
- Otimizar uma cláusula do comando **for** ;
- Utilizar o comando **do-while** ;
- Criar um aninhamento de Repetições ;
- Utilizar o comando **break** ;
- Utilizar o comando **continue** ;
- Utilizar o comando **goto** ;

# Principal Objetivo

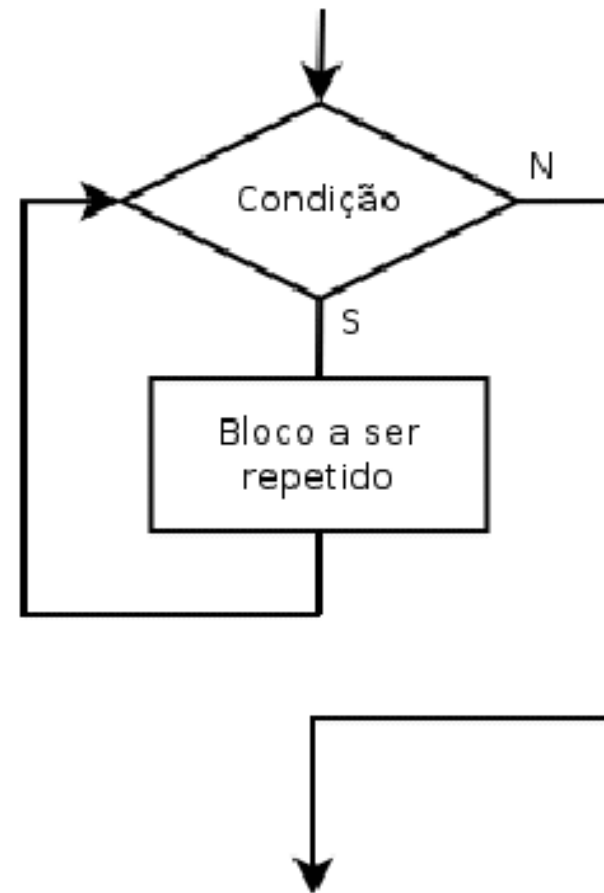
Apresentar comandos de repetição em C.

# Laços de Repetição

Permite a execução de um bloco de comandos por um número de vezes que uma determinada condição é satisfeita.

# Repetição por Condição.

```
enquanto condição faça  
    sequência de comandos ;  
fim enquanto
```



# O Comando Enquanto

## ■ Pseudocódigo

*Leia A e B ;*

***enquanto***  $A < B$

*A recebe  $A + 1$  ;*

*Imprima (A);*

***Fim enquanto***

# Laço Infinito

- Laço Infinito ou *Loop Infinito* ;
  - *Não definimos uma Condição de Parada ;*
  - *A condição de parada existe, mas nunca é atingida ;*



# Laço Infinito

```
// Condição Errônea  
x := 4 ;  
enquanto (x < 5) faça  
    x := x - 1;  
    imprima (x)  
fim enquanto
```

```
// Não Muda Valor  
x := 4 ;  
enquanto (x < 5) faça  
    imprima (x)  
fim enquanto
```

# Comando *while*

- Equivale ao comando “enquanto”

```
while (condição) {  
    sequência de comandos ;  
}
```

# Comando *while*

- O teste da condição é feito no início, antes de executar.

```
while (condição/expressao) {  
    bloco_de_comandos;  
}
```

- *Enquanto a condição/expressão for verdadeira (qualquer valor diferente de zero), será executado o bloco\_de\_comandos; caso contrário (condição for falsa), executará o primeiro comando após o while.*
- *Uso: Quando é necessário repetir um bloco de comandos por um número de vezes.*

# Comando *while*

- Exemplo:

```
int i = 0;
while (i < 10) {
    printf("%d - Olá Mundo\n", i);
    i++;
}
```

- Enquanto *i* for menor que 10, a mensagem “*i* – Olá Mundo” será escrita na tela.
- Após isso irá continuar a execução a partir do próximo comando.

# Comando *while*

- Observações:

- O comando `while` segue todas as recomendações definidas para *if* quanto ao uso de chaves e da condição usada ;
- Colocar o “;” após o comando `while` faz com que o compilador entenda que o comando `while` já terminou e trate o próximo comando como se estivesse fora do `while` ;
- É responsabilidade do programador modificar valores para não entrar em laço infinito ;

# Comando *while*

## ■ Exemplo:

```
1  #include<stdio.h>
2
3  int main(void){
4      int a, b;
5
6      printf("Digite o valor de A: ") ;
7      scanf("%d", &a) ;
8
9      printf("Digite o valor de B: ") ;
10     scanf("%d", &b) ;
11
12     while (a < b) {
13         a = a + 1 ;
14         printf ("%d \n", a) ;
15     }
16
17     return 0 ;
18 }
19
```

```
Digite o valor de A: 3
Digite o valor de B: 10
4
5
6
7
8
9
10
```

# Comando *while*

- Atividade 1:

- Faça um programa que leia um número inteiro positivo  $N$  e imprima todos os números naturais de 0 até  $N$  (inclusive) em ordem crescente.

Faça um programa que leia um número inteiro positivo  $N$  e imprima todos os números naturais de  $N$  até 0 (inclusive) em ordem decrescente.

Faça um programa que leia um número inteiro positivo  $N$  e imprima todos os números PARES de 1 até  $N$  (inclusive).

# O Comando *for*

- Muito similar ao comando *while* ;
- Uma variável de controle é utilizada (repetição por contagem)

- Forma geral:

```
for (inicialização ; condição ; incremento) {  
    sequência de comandos ;  
}
```

- Executará o bloco\_de\_comandos enquanto o valor da variável de controle não exceder o valor máximo determinado ;
- Uso: Quando é necessário repetir um bloco de comandos por um número determinado de vezes ;



# O Comando *for*

## Exemplo:

```
1  #include<stdio.h>
2
3  int main(void){
4      int a, b, c;
5
6      printf("Digite o valor de A: ") ;
7      scanf("%d", &a) ;
8
9      printf("Digite o valor de B: ") ;
10     scanf("%d", &b) ;
11
12     for (c = a ; c <= b ; c++ ) {
13         printf ("%d \n", c) ;
14     }
15
16     return 0 ;
17 }
```

```
Digite o valor de A: 3
Digite o valor de B: 10
3
4
5
6
7
8
9
10
```

# *while x for*

```
1  #include<stdio.h>
2
3  int main(void){
4      int i, s = 0 ;
5
6      for (i = 1 ; i <= 10 ; i++ ) {
7          s = s + i ;
8      }
9
10     printf("Soma = %d \n", s) ;
11
12     return 0 ;
13 }
```

```
1  #include<stdio.h>
2
3  int main(void){
4      int i, s = 0 ;
5
6      i = 1 ;
7      while (i <= 10) {
8          s = s + i ;
9          i++ ;
10     }
11
12     printf("Soma = %d \n", s) ;
13
14     return 0 ;
15 }
```

# Atividade – Usando for

```
1 // Programa 1.
2 #include<stdio.h>
3 #include<stdlib.h>
4
5 int main()
6 {
7     int i ;
8
9     for (i = 500; i <=550 ; i++)
10    {
11        printf(" %d", i);
12    }
13
14    system("PAUSE");
15
16 }
```

```
1 // Programa 1.
2 #include<stdio.h>
3 #include<stdlib.h>
4
5 int main()
6 {
7     int i ;
8
9     for (i = 1; i <=100 ; i++)
10    {
11        if (i % 2 != 0 )
12        {
13            printf(" %d", i);
14        }
15    }
16
17    system("PAUSE");
18
19 }
```

# Comando *for*

- Atividade 1:
  - Faça um programa que leia um número inteiro positivo  $N$  e imprima todos os números naturais de 0 até  $N$  em ordem crescente.
  - Faça um programa que leia um número inteiro positivo  $N$  e imprima todos os números naturais de  $N$  até 0 em ordem decrescente.

# Omitindo uma clausula no *for*

- Dependendo da situação podemos omitir a:
  - Inicialização ;
  - Condição ;
  - Incremento ;

*Independente de qual clausula for omitida, o comando for exige que se coloquem o dois operadores “;”*

# Omitindo uma clausula no *for*

- Inicialização ;

```
1  #include<stdio.h>
2
3  int main(void){
4      int i = 2, s = 0 ;
5
6      for ( ; i <= 10; i++) {
7          s = s + i ;
8      }
9
10     printf("Soma = %d \n", s) ;
11
12     return 0 ;
13 }
```

# Omitindo uma clausula no *for*

- Condição

for( inicialização ; \_\_\_\_ ; incremento) ;

- Não existindo a condição ela sempre será verdadeira;
- Loop Infinito ;

# Omitindo uma clausula no *for*

## - Incremento ;

```
1  #include<stdio.h>
2
3  int main(void){
4      int i = 2, s = 0 ;
5
6      for ( ; i <= 10; ) {
7          s = s + i ;
8          printf("%d Iteração - Soma: %d \n ", i, s);
9          i = i + 1 ;
10     }
11
12     printf("Soma = %d \n", s) ;
13
14     return 0 ;
15 }
```



# Usando o operador “,” no *for*

- Em linguagem C o operador “,” é um separador de comandos ;
- Ele permite determinar uma lista de expressões que devem ser executadas sequencialmente, inclusive dentro do comando **for** ;

```
1  #include<stdio.h>
2
3  int main(void){
4      int i, j ;
5
6      for ( i = 0, j = 100 ; i < j ; i++, j-- ) {
7          printf("i = %d - j = %d . \n ", i, j);
8      }
9
10
11
12     return 0 ;
13 }
14
```

```
i = 32 - j = 68 +
i = 33 - j = 67 +
i = 34 - j = 66 +
i = 35 - j = 65 +
i = 36 - j = 64 +
i = 37 - j = 63 +
i = 38 - j = 62 +
i = 39 - j = 61 +
i = 40 - j = 60 +
i = 41 - j = 59 +
i = 42 - j = 58 +
i = 43 - j = 57 +
i = 44 - j = 56 +
i = 45 - j = 55 +
i = 46 - j = 54 +
i = 47 - j = 53 +
i = 48 - j = 52 +
i = 49 - j = 51 +
```

# O comando *do-while*

- Bastante semelhante ao *while* ;
- Testa a condição no final ;
- O comando *do-while* é utilizado sempre que se desejar que a sequência de comandos seja executada pelo menos uma vez
- Forma geral:  
    *do {*  
        *sequência de comandos ;*  
    *} while (condição) ;*

# O comando *do-while*

- Diferente do if-else, é necessário colocar um “;” depois da condição do comando do-while ;
- *O comando do-while segue todas as recomendações definidas para o comando if para o uso de chaves {} e definição da condição usada ;*
- *Programador, tome cuidado com laços infinitos!*

# O comando *do-while*

## ■ Exemplo:

```
1  #include<stdio.h>
2
3  int main(void){
4      int x = 0;
5
6      do {
7          x++;
8          printf("Iteração %d \n", x) ;
9      } while (x <= 10);
10
11
12      return 0 ;
13 }
```

```
Iteração 1
Iteração 2
Iteração 3
Iteração 4
Iteração 5
Iteração 6
Iteração 7
Iteração 8
Iteração 9
Iteração 10
Iteração 11
```

# O comando *do-while*

## ■ Exemplo:

```
1  #include<stdio.h>
2
3  int main(void){
4      int x;
5
6      do {
7          printf("Escolha uma opção:\n");
8          printf("(1) Opção 1\n");
9          printf("(2) Opção 2\n");
10         printf("(3) Opção 3\n");
11         scanf("%d", &x) ;
12     }while((x < 1) || (x > 3 ));
13
14     printf("Você escolheu a opção %d. \n", x);
15
16     return 0 ;
17 }
18
```

```
Escolha uma opção:
(1) Opção 1
(2) Opção 2
(3) Opção 3
9
Escolha uma opção:
(1) Opção 1
(2) Opção 2
(3) Opção 3
0
Escolha uma opção:
(1) Opção 1
(2) Opção 2
(3) Opção 3
-1
Escolha uma opção:
(1) Opção 1
(2) Opção 2
(3) Opção 3
3
Você escolheu a opção 3.
```

# O comando *do-while*

## ■ Exemplo:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void){
5      int x, num, cont = 0;
6
7      num = rand() % 10 ;
8      do {
9          printf("Digite um número:\n");
10         cont++;
11         scanf("%d", &x) ;
12     }while(x != num);
13
14     printf("Você Acertou em %d tentativas. \n", cont);
15
16     return 0 ;
17 }
```

# Contadores e Somadores

- Contagens, somas e multiplicações acumulativas são comuns.
- Ex.: Contabilizar a quantidade de números pares dentro de um intervalo.
- Ex.: média, totais.

# Contadores

- Contadores e somadores tem como valor inicial, o zero (elemento neutro da soma) e, vão sendo acrescidos por mais um termo no decorrer da execução (o valor um é bem comum no caso de contadores) e diferentes valores quando se trata de somadores.
- Variáveis utilizadas para o cálculo de produtórios são inicializadas (usualmente) com o valor **1 (um)** (elemento neutro da multiplicação) e são atualizadas com o resultado do seu valor corrente multiplicado por um novo termo.



# Atividade – Contadores

- Faça um programa em C usando for, que SOME o valor de todos os número ímpares entre 1 e 100.
- Ex:  $1 + 3 + 5 + 7 + \dots + 99 \rightarrow ?$

# Aninhamento de Repetições

- Uma repetição aninhada é simplesmente um comando de repetição dentro do bloco de outro comando de repetição.
- Semelhante ao que é feito com o comando if ;
- Forma geral:

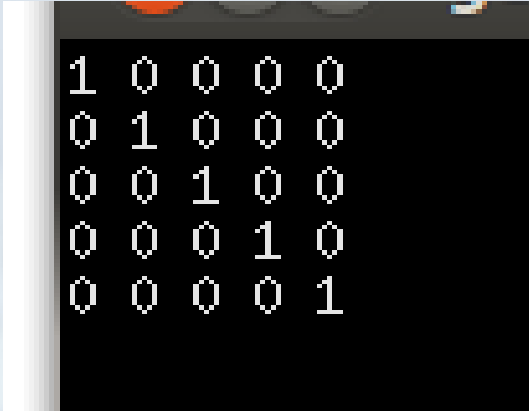
```
repetição (condição 1) {  
    sequência de comandos ;  
    repetição (condição 2) {  
        sequência de comandos ;  
        Repetição ...  
    }  
}
```

# Aninhamento de Repetições

- É útil para percorrer matrizes ;
- Útil também quando um único comando não é suficiente para efetuar a tarefa ;
- A linguagem C não proíbe que misturem comandos de repetições de tipos diferentes no aninhamento de repetições ;

# Aninhamento de Repetições

```
1  #include <stdio.h>
2
3  int main(void){
4      int i, j;
5
6      for (i = 0 ; i < 5 ; i++) {
7          for (j = 0 ; j < 5 ; j++){
8              if (i == j)
9                  printf("1 ");
10             else
11                 printf("0 ");
12         }
13         printf("\n");
14     }
15
16     return 0 ;
17 }
```



|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 |

# Atividade

- Imprimir em formato de Matriz os números de 1 até 1000. Ou seja uma “matriz” 100 x 100.

```
001 002 003 004 005 006 007 008 009 010
011 012 013 014 015 016 017 018 019 020
021 022 023 024 025 026 027 028 029 030
031 032 033 034 035 036 037 038 039 040
041 042 043 044 045 046 047 048 049 050
051 052 053 054 055 056 057 058 059 060
061 062 063 064 065 066 067 068 069 070
071 072 073 074 075 076 077 078 079 080
081 082 083 084 085 086 087 088 089 090
091 092 093 094 095 096 097 098 099 100
```

# O comando break

- Já usamos break no switch
- Pode ser utilizado para interromper a execução em qualquer comando de repetição (*for*, *while*, *do-while*)
- O comando *break* é utilizado para terminar de forma abrupta uma repetição. Por exemplo, se estivermos dentro de uma repetição e determinado resultado ocorrer, o programa deverá sair da repetição e continuar na primeira linha seguinte a ela ;

# O comando break

```
1  #include <stdio.h>
2
3  int main(void){
4      int x, i ;
5
6      printf("Digite um Número: ");
7      scanf("%d", &x);
8
9      for (i = 0 ; i < 100 ; i++) {
10         if (i == x )
11             break ;
12         else
13             printf("Não é esse o número - Iteração %d \n", i);
14     }
15
16     printf("O Número Digitado foi %d \n", x);
17     printf("Você economizou %d Iterações \n", 100 - i);
18
19     return 0 ;
20 }
```

```
Digite um Número: 10
Não é esse o número - Iteração 0
Não é esse o número - Iteração 1
Não é esse o número - Iteração 2
Não é esse o número - Iteração 3
Não é esse o número - Iteração 4
Não é esse o número - Iteração 5
Não é esse o número - Iteração 6
Não é esse o número - Iteração 7
Não é esse o número - Iteração 8
Não é esse o número - Iteração 9
O Número Digitado foi 10
Você economizou 90 Iterações
```

# O comando break

- O comando break deverá sempre ser colocado dentro de um comando if ou else que esta dentro da repetição ;
- break ;



# O comando *continue*

- Similar ao comando *break* ;
- Quando executado o comando *continue*, o comando interrompe apenas aquela repetição e passa para a próxima repetição do laço, se ela existir ;
- Sempre deverá ser usado dentro de um *if* ou *else* que esta dentro da repetição ;

# O comando *continue*

```
1  #include <stdio.h>
2
3  int main(void){
4      int x, i ;
5
6      printf("Digite um Número: ");
7      scanf("%d", &x);
8
9      for (i = 0 ; i < 100 ; i++) {
10         if (i == x )
11             continue ;
12         printf("0 número é: %d \n", i);
13     }
14
15     //printf("0 Número Digitado foi %d \n", x);
16     //printf("Você economizou %d Iterações \n", 100 - i);
17
18     return 0 ;
19 }
```

```
Digite um Número: 10
0 número é: 0
0 número é: 1
0 número é: 2
0 número é: 3
0 número é: 4
0 número é: 5
0 número é: 6
0 número é: 7
0 número é: 8
0 número é: 9
0 número é: 11
0 número é: 12
0 número é: 13
0 número é: 14
0 número é: 15
0 número é: 16
0 número é: 17
0 número é: 18
0 número é: 19
0 número é: 20
0 número é: 21
0 número é: 22
0 número é: 23
```

# O comando *goto*

O comando é um salto condicional para um local especificado por uma palavra-chave no código.

- Pode ser substituído por outros ;
- Forma geral:  
*destino:*  
*goto destino ;*

# O comando *goto*

```
1  #include <stdio.h>
2
3  int main(void){
4      int x, i ;
5
6      printf("Digite um Número: ");
7      scanf("%d", &x);
8
9      for (i = 0 ; i < 100 ; i++) {
10         if (i == x )
11             goto fim;
12         printf("O número é:  %d \n", i);
13     }
14
15     fim:
16     printf("O Número Digitado foi %d \n", x);
17     printf("Você economizou %d Iterações \n", 100 - i);
18
19     return 0 ;
20 }
21
```

# O comando *goto*

```
1  #include <stdio.h>
2
3  int main(void){
4      int i = 0 ;
5      // laço de repetição com goto
6      inicio:
7      if ( i < 5){
8          printf("Número %d \n", i);
9          i++ ;
10         goto inicio ;
11     }
12
13     return 0 ;
14 }
```

# Dúvidas, Críticas ou Sugestões

[diegobertolini@utfpr.edu.br](mailto:diegobertolini@utfpr.edu.br)